CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

3F WrappedAsset

0x2dad4FCD26e0902661cBd80f03DDB5D5908DF340

Ethereum

Compiler: v0.8.34 (implementation behind ERC1967 proxies)

Date: October 12, 2026

OVERALL RISK RATING AI-assisted audit. Not a substitute for manual review.

High

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. 3F wraps underlying assets one to one through WrappedAsset instances deployed as ERC1967 proxies by a Solady factory, with roughly $41M of underlying held across them. The contract itself is well built: it uses ERC-7201 namespaced storage so proxy upgrades cannot collide, calls _disableInitializers in the constructor, builds on Solady's audited ERC20 and OwnableRoles, and, importantly, leaves the burn path free of any role check so a holder can always redeem to the underlying even if every transfer role is revoked. That exit design is better than most wrappers of this kind. The risk is not in the Solidity, it is in how the system is deployed and what the wrapper assumes about its underlying. Every proxy observed shares one admin address, and that address is an externally owned account with no code, so a single private key can replace the implementation behind every wrapper holding user funds. Separately, the mint path credits the requested amount rather than the amount actually received, so any underlying that charges a transfer fee or rebases downward quietly leaves the wrapper unable to honour the last redemptions.

Findings (8)

F-001

High

All wrapper proxies share one admin, and that admin is an externally owned account

deployment configuration (on-chain)

DESCRIPTION

Every WrappedAsset is deployed through the Solady ERC1967Factory at 0x54f862...e82e, and each Deployed event observed carries the same admin, 0xabFF377a1F7a0B061c9D4395b53f1488B153333f. Querying eth_getCode for that address returns 0x, so it is an externally owned account rather than a multisig or a timelock. In the ERC1967Factory model the admin can call upgrade at any moment, with no delay and no second signature, for every proxy it administers.

IMPACT

One private key can replace the implementation behind every wrapper holding underlying assets. A replacement implementation can mint wrapper tokens without depositing, remove the burn path that currently guarantees redemption, or transfer the pooled underlying out directly. The protocol's own TVL methodology measures underlying held by exactly these proxies, so the full balance is in scope. This is a key compromise and governance risk rather than an exploitable bug, but it is the single assumption the entire system rests on, and today it rests on one key with no delay behind it.

RECOMMENDATION

Move the proxy admin to a multisig, and put a timelock in front of upgrades so holders can observe a pending implementation change and redeem before it lands. Publish the admin address and the timelock delay where users can find them, since right now the only way to learn any of this is to read the deployment events. If upgradeability is no longer needed for a given wrapper, renouncing the admin for that proxy is stronger than any policy statement.

```
// Deployed events from the factory, all sharing one admin:
// proxy 0x86b495e4cb00ab18ad94bfd7920479cc79e8ebfe
// proxy 0x4e013ca8ff612a58f53c822904cdd0ec538a4a4f
// admin 0xabFF377a1F7a0B061c9D4395b53f1488B153333f
```

```
//
// eth_getCode(admin) == "0x"  ->  externally owned account
```

F-002

Medium

Mint credits the requested amount instead of the amount actually received

src/funds/WrappedAsset.sol:110–111

DESCRIPTION

mint pulls the underlying with safeTransferFrom and then mints exactly amount, without measuring the contract's balance before and after. For a plain ERC20 the two are identical. For an underlying that charges a transfer fee, that takes a cut in a hook, or that rebases downward, the wrapper receives less than it credits, and nothing in the contract notices. The factory places no constraint on which asset a wrapper may be created for, so the safety of this assumption depends entirely on operational discipline at deployment time.

IMPACT

The one to one backing that the wrapper's entire design promises is broken by exactly the shortfall. Because burn pays out the full amount requested, the losses land on whoever redeems last: early redeemers are paid in full and the final holders find the underlying exhausted. The failure is silent, there is no event or invariant check that would surface it before someone cannot withdraw.

RECOMMENDATION

Measure the received amount rather than trusting the requested one: read the underlying balance before and after the transfer and mint the difference. If fee bearing and rebasing assets are intentionally out of scope, enforce that at deployment, for example by rejecting an underlying whose balance change does not match a probe transfer, and state the restriction in the documentation.

```
function mint(address to, uint256 amount) external override {
to.checkNotZero();
if (to != msg.sender) _checkRoles(ISSUER_ROLE);
WrappedAssetStorage storage _storage = _wrappedAssetStorage();
_storage.underlying.safeTransferFrom(msg.sender, address(this), amount);
_mint(to, amount);   // credits `amount`, not what actually arrived
}
```

F-003

Medium

Transfer restrictions are owner controlled and can strand tokens held by contracts

src/funds/WrappedAsset.sol:166–167

DESCRIPTION

A transfer succeeds only when the sender holds SENDER_ROLE or the receiver holds RECEIVER_ROLE, and both roles are granted and revoked by the owner at will with no delay. Redemption is correctly exempted, since burn passes address(0) as the destination and the check short circuits, so an ordinary holder can always exit to the underlying. The gap is a contract that receives wrapper tokens: if it has neither role and cannot call burn itself, the balance cannot move at all.

IMPACT

Any integration that accepts wrapper tokens without implementing a burn path can be permanently locked out of its balance by a role change it does not control. An owner can also freeze all secondary movement at

any moment by revoking roles, which is a legitimate compliance tool but is indistinguishable, from the outside, from a unilateral freeze.

RECOMMENDATION

Document the role policy publicly, including who holds the owner key and under what circumstances roles are revoked, so integrators can price the risk. Emit events on role changes so monitoring is possible, and consider a timelock on revocation so an integrating protocol has a window to exit rather than discovering the change when a transaction reverts.

```
function _beforeTokenTransfer(address from, address to, uint256 amount) internal
override {
if (from != address(0) && to != address(0) && !hasAnyRole(to, RECEIVER_ROLE) &&
!hasAnyRole(from, SENDER_ROLE)) {
revert Unauthorized();
}
super._beforeTokenTransfer(from, to, amount);
}
```

F-004

Low

Every proxy depends on being initialized in the same transaction it is deployed

src/funds/WrappedAsset.sol:72–73

DESCRIPTION

The implementation correctly calls _disableInitializers in its constructor, so the implementation itself cannot be seized. The proxies rely instead on initialize being called atomically at deployment, which the factory's deployAndCall path does. Any proxy created through the plain deploy path, or whose initialising call reverted while the deployment succeeded, is left uninitialised and can be initialised by the first caller, who then becomes owner with full role control over that wrapper.

IMPACT

Limited to deployment mistakes rather than ongoing operation, but the consequence of one is total control of the affected wrapper. The window is also silent: an uninitialised proxy looks like a normal contract until someone claims it.

RECOMMENDATION

Confirm that every proxy already deployed has a non zero owner, which takes one call per wrapper, and make the deployment script fail loudly if initialisation is not bundled into the same transaction. Monitoring for an Initialized event on each new proxy closes the gap permanently.

```
constructor() {
_disableInitializers();   // protects the implementation, not the proxies
}

function initialize(address owner_, address initialIssuer_, address underlying_, string
calldata symbol_, string calldata name_)
public initializer { ... }
```

F-005

Low

decimals is read from the underlying on every call rather than cached

src/funds/WrappedAsset.sol:155–156

DESCRIPTION

decimals performs an external staticcall to the underlying each time it is read. If the underlying does not implement decimals the call reverts, which breaks every integrator that queries it, and if the underlying is itself upgradeable its decimals could in principle change after wrappers and integrations are already live.

IMPACT

A reverting decimals makes the wrapper unusable by most front ends and aggregators even though wrapping and redemption still work. A changing value would silently reprice every integration that cached it. Both scenarios depend on the choice of underlying, which is an operational decision made per wrapper.

RECOMMENDATION

Read the underlying's decimals once during initialize, store it in the namespaced storage struct, and return the stored value. This fixes the value at the moment the wrapper is created, removes an external call from a hot view function, and makes the behaviour independent of the underlying's future.

```
function decimals() public view override returns (uint8) {
return ERC20(_wrappedAssetStorage().underlying).decimals();
}
```

F-006

Low

Initialization emits no event recording the underlying

src/funds/WrappedAsset.sol:87–94

DESCRIPTION

initialize sets the symbol, name and underlying but emits nothing. The underlying is readable through a view function, but there is no log entry marking which asset a given wrapper was bound to or when, so reconstructing the history of the system requires calling every proxy rather than reading events.

IMPACT

Operational and forensic rather than financial. Indexers, dashboards and incident response all have to enumerate proxies and query them individually instead of subscribing to a single event stream.

RECOMMENDATION

Emit an initialisation event carrying the underlying, owner and initial issuer. The factory's Deployed event already provides the proxy and admin, so one event here completes the picture for anyone tracking the system from logs alone.

```
function initialize(...) public initializer {
_storage.symbol = symbol_;
_storage.name = name_;
_storage.underlying = underlying_;
_initializeOwner(owner_);
_setRoles(initialIssuer_, ISSUER_ROLE | SENDER_ROLE);
// no event
}
```

F-007

Informational

Upward rebases and donations accumulate with no claim path

src/funds/WrappedAsset.sol:121–126

DESCRIPTION

The wrapper tracks backing implicitly through its own underlying balance. If the underlying rebases upward, or if anyone transfers underlying directly to a wrapper, the surplus sits in the contract. Redemption pays out exactly the burned amount, so the excess is never distributed and no function can sweep it.

IMPACT

The surplus strengthens the backing, which is harmless, but it is permanently inaccessible and it makes the reported TVL diverge from the amount actually redeemable by holders.

RECOMMENDATION

Decide explicitly whether surplus belongs to holders or to the protocol, and implement whichever answer is intended. Leaving it undefined means the question gets settled by whoever notices first, which is rarely the outcome anyone planned.

```
function burn(address from, address to, uint256 amount) external override {
...
_burn(from, amount);
_storage.underlying.safeTransfer(to, amount);   // exactly `amount`, surplus
untouched
}
```

F-008

Informational

No audit is listed publicly for a system holding roughly $41M

public listing

DESCRIPTION

DefiLlama shows no audit for this protocol, and no report is linked from the project site at the time of this review. This observation is about the public record rather than the code: a review may well exist and simply not be published. The distinction matters because users and integrators check the listing, not the team's internal history.

IMPACT

Prospective depositors and integrating protocols have no way to judge whether the system has been reviewed. For a wrapper holding other people's assets behind an upgradeable proxy, that absence carries more weight than it would for a simpler contract.

RECOMMENDATION

If a review exists, link it from the site and update the DefiLlama listing, which takes minutes and removes an avoidable question. If one does not, the proxy admin configuration in F-001 is the place to start, since it is the finding that governs every other guarantee in the system.

```
// defillama.com/protocol/3f  ->  audits: "0" (none listed)
```

Gas Optimizations

Cache the storage pointer once per function

src/funds/WrappedAsset.sol:113–127

Several functions call _wrappedAssetStorage() and then read a single field. The helper is cheap, but taking the pointer once and reading through it keeps the pattern uniform and avoids repeated slot computation in functions that touch more than one field.

Store decimals at initialisation

src/funds/WrappedAsset.sol:155–156

Beyond the correctness benefit described in F-005, caching decimals removes an external call from a view function that front ends and aggregators call constantly. The saving is per read rather than per transaction, which adds up across integrations.

Short circuit the transfer hook for role holders first

src/funds/WrappedAsset.sol:166–167

The hook evaluates two zero address comparisons and two role lookups on every transfer. Checking the sender's SENDER_ROLE first, which is the common case for protocol contracts, lets the usual path exit after a single lookup.

Appendix · Metadata

Contract	3F WrappedAsset
Address	0x2dad4FCD26e0902661cBd80f03DDB5D5908DF340
Chain	Ethereum
Compiler	v0.8.34 (implementation behind ERC1967 proxies)
Audit ID	independent-3f
Date	10/12/2026

Files analyzed

```
[user] src/funds/WrappedAsset.sol
[user] src/libs/common/LibChecks.sol
[user] src/interfaces/funds/IWrappedAsset.sol
[dependency] ERC1967Factory (Solady) 0x54f862fa0612a8709f6dec4a7b39af015cd4e82e
```

MANUAL AUDIT UPGRADE

Need a deeper audit?

$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

Request Manual Audit ’

https://cryptohawking.com/audit/manual?ref=independent-3f

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

https://cryptohawking.com/services

WhatsApp (preferred): +92 322 4274236, https://wa.me/923224274236